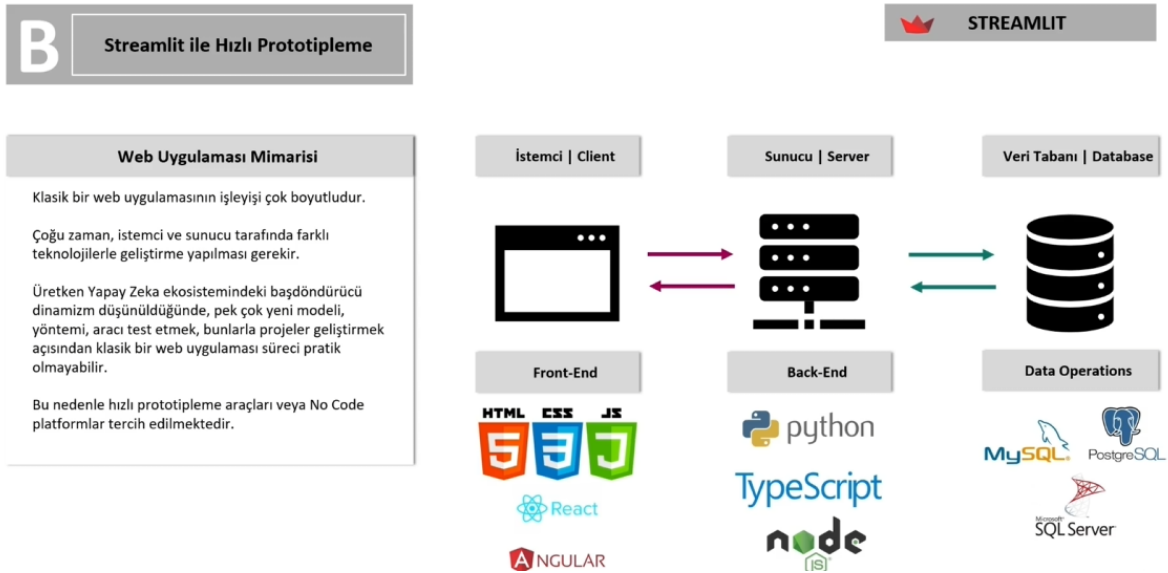


Üretken yapay zeka eğitimimizin hazırlık safhasında artık çok kritik bir viraja girdik. Çalışma ortamımızı kurduk, sanal ortamlarımızı yapılandırdık; şimdi ise bu modelleri son kullanıcıyla buluşturacağımız vitrini, yani **Streamlit** ile hızlı prototipleme sürecini ele alacağız.

Yapay zeka modellerini sadece bir terminal ekranında çalıştırmak yerine, onları şık ve işlevsel bir web arayüzüne dönüştürmek, projelerinizin değerini katlayacaktır. Gelin, neden klasik yöntemler yerine Streamlit'i tercih ettiğimizi ve bu mimarinin bize neler sunduğunu inceleyelim.

1. Klasik Web Mimarisi ve Geliştirme Bariyerleri

Geleneksel bir web uygulaması geliştirmek istiyorsanız, genellikle üç katmanlı bir mimariyle (İstemci-Sunucu-Veri Tabanı) baş başa kalırsınız.



- **Frontend (Ön Yüz):** Kullanıcının gördüğü kısımdır. HTML, CSS ve JavaScript temelinde yükselir; React veya Angular gibi karmaşık çerçeveler (framework) gerektirir.
- **Backend (Arka Yüz):** Uygulamanın asıl mantığının (Business Logic) çalıştığı, sunucu tarafı süreçlerdir. Python, Node.js veya TypeScript gibi dillerle yönetilir.
- **Veri İşlemleri:** Sunucunun veri tabanı (MySQL, PostgreSQL vb.) olan etkileşimini kapsar.

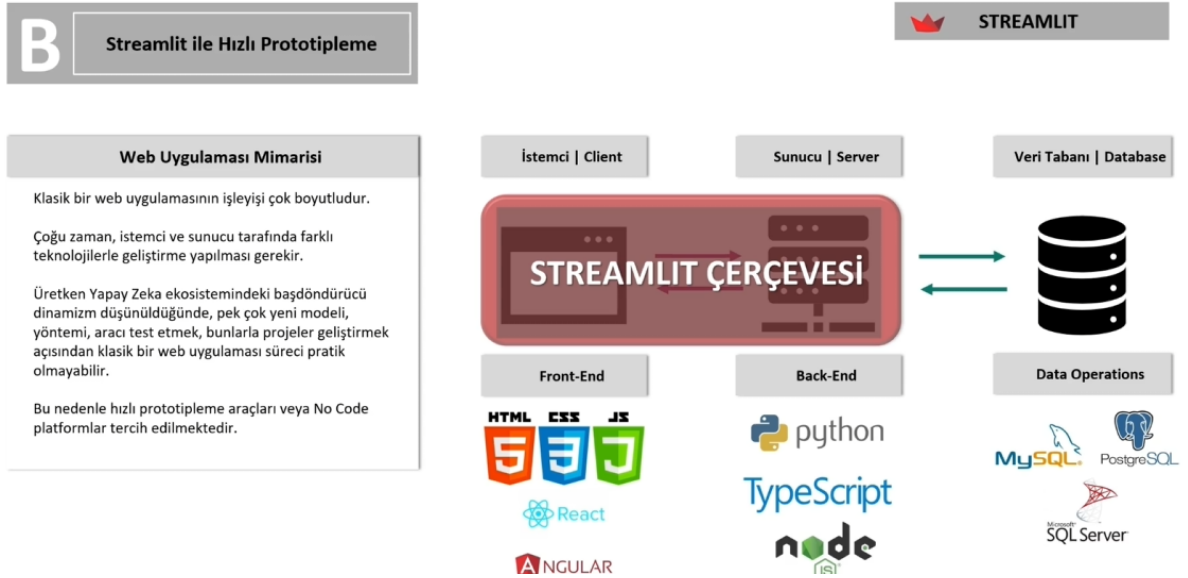
Eğer halihazırda "Full-stack" bir geliştirici değilseniz, sadece bir yapay zeka modelini test etmek için tüm bu teknolojileri öğrenmeye çalışmak, asıl odağınızdan sapmanıza ve aylar süren bir öğrenme sürecine girmenize neden olabilir.

2. Neden Hızlı Prototipleme?

Yapay zeka ekosistemi, 2026 yılı itibarıyla baş döndürücü bir dinamizme sahip. Her ay yeni modellerin, yöntemlerin ve araçların çıktığı bu ortamda, bir fikri test etmek için klasik web geliştirme süreçleriyle vakit kaybetmek pratik değildir. Bu noktada imdadımıza **Hızlı Prototipleme** araçları ve **No-Code** platformlar yetişir.

3. Streamlit: Python ile Tek Çatı Altında Geliştirme

Streamlit, yapay zeka ve makine öğrenmesi toplulukları tarafından en çok tercih edilen çerçevelerden biridir. Streamlit'in sunduğu en büyük devrim, **hem ön yüzü hem de arka yüzü tek bir Python dosyası içinde** kodlayabilmenize olanak tanımasıdır.



- **Ayricalığı:** HTML veya JavaScript bilmenize gerek kalmadan, sadece Python fonksiyonları kullanarak profesyonel görünen arayüzler oluşturabilirsiniz.
- **Entegrasyon:** Veri operasyonlarında kullanılan hemen hemen tüm kütüphanelerle uyumludur; dolayısıyla veri tabanı süreçlerinden kopmanıza gerek kalmaz.

4. Ekosistemdeki Diğer Seçenekler

Streamlit bu alandaki tek oyuncu değildir. İhtiyacınıza göre değerlendirebileceğiniz alternatifler de mevcuttur:

B

Streamlit ile Hızlı Prototipleme



STREAMLIT



- **Gradio:** Özellikle model demoları için oldukça popüler bir diğer çerçevedir.
- **Vercel ve Replit:** Sadece bir çerçeve değil, aynı zamanda bu uygulamaları kolayca yayınlayabileceğiniz (deployment) platformlar sunarlar.

Biz bu eğitimde, kullanım kolaylığı ve çok boyutlu yapısı nedeniyle Streamlit üzerinden ilerleyeceğiz.

5. Temel Operasyonlara Hazırlık

Web arayüzümüzü Streamlit ile hızlıca ayağa kaldırmayı öğrendikten sonra, eğitimin başında bahsettiğimiz o 5 temel operasyonu bu arayüzlere entegre etmeye başlayacağız.

A

Çalışma Ortamının Yapılandırılması

B

Streamlit ile Hızlı Prototipleme

01

Metin Üretme

02

Görsel Üretme

03

Ses Üretme

04

Kod Üretme

05

Çoklu-Formla Çalışma

1. **Metin Üretme**
2. **Görsel Üretme**

3. **Ses Üretme**
4. **Kod Üretme**
5. **Çoklu-Formla Çalışma**